ChoEn: Choreography Enforcement in the Zero Trust Era via Smart Contracts

Francesco Spegni[1] Marco Bernardo[1] Luca Spalazzi[2]

[1] Università degli Studi di Urbino, Urbino, Italy
[2] Università Politecnica delle Marche, Ancona, Italy

Abstract. When independent organizations cooperate in joint projects, business processes must conform to choreographies previously agreed upon by the organizations themselves. Such choreographies, expressed as multiparty session types or BPMN choreography diagrams, constrain admissible interaction patterns while leaving each partner autonomous in implementing its internal processes. Ensuring conformance in such federated scenarios is challenging since organizations may change their processes and re-deploy them throughout the project, making synthesis-based correct-by-construction approaches insufficient. We present ChoEn, a tool for generating decentralized runtime enforcers from a given choreography. Enforcers generated through it intercept requests to execute critical actions and dynamically grant, postpone, or deny them according to the choreography and the current execution trace. The generated monitors also achieve transparency, ensuring that behaviors of conformant processes are not altered, and soundness, guaranteeing that violating behaviors are instead detected and amended. ChoEn generates monitors that do not rely on trusted intermediaries, since they are encoded as Solidity smart contracts deployable on distributed ledger technologies. This realizes a decentralized approach to enforcement, providing inspectability of the enforcer state as well as non-repudiability and accountability of the monitor decisions. A testbed of choreographies given as multiparty session types is used to generate Solidity enforcers that are assessed by using public Ethereum testnets such as Sepolia.

1 Introduction

Business processes are rarely developed in isolation. When multiple organizations join forces to deliver a large-scale project, each partner typically retains ownership of the internal implementation of its business processes, while the parties jointly agree on how their processes must interact at a relatively small number of well-defined *contact points*. Such agreements are commonly expressed through *choreographies* [30,29], which are high-level specifications describing admissible patterns of interaction among the participants of a federation. Unlike orchestrations or executable workflow models, choreographies are not themselves executable artifacts. Rather, they describe from a global viewpoint which sequences of message exchanges are legitimate, without prescribing how each participant should internally realize its share of the interaction. A choreography therefore

acts as a *contract*: it fixes the terms of collaboration among independent organizations while leaving the internal organization of work to each party [8].

A central problem in this setting is how to guarantee that the executable business processes deployed by participating organizations actually conform to the subscribed choreography. A prominent line of work addresses this problem through *realizability* [4,19]: determining whether a choreography admits a decomposition into local executable processes whose composed behavior matches the global specification. When realizability holds, (stubs of) executable processes can be automatically synthesized from the choreography, yielding implementations that are correct by design with respect to the agreed contract.

While elegant, this synthesis-based methodology relies on assumptions that do not always hold in practice, particularly in industrial settings characterized by low or no mutual trust among the involved organizations. Federations may be assembled *opportunistically*, with organizations having little or no prior working relationship joining forces to bid for and deliver a project. In such settings, each organization typically retains full autonomy in the design of its own business processes. Thus, expecting organizations to implement the synthesized processes faithfully does not provide a persistent guarantee of conformance. Organizations may modify the synthesized processes, immediately or over time, to accommodate internal constraints that the choreography does not capture (e.g., resource allocation with respect to concurrent projects, understaffing, personnel turnover, and so on). Once a synthesized process enters the boundary of an autonomous organization, its conformance can therefore no longer be assumed. Thus, an initial policy compliance is not guaranteed to be maintained over time. When nothing can be taken for granted, every single event must be monitored, always giving as few privileges as possible. A system designed on the basis of these assumptions is called a *zero-trust architecture* (ZTA) [32].

In this paper we present ChoEn (https://gitlab.com/fspegni/choen), a tool that, when operating in a ZTA, helps the automated synthesis of enforcing monitors that can be attached to running processes so as to ensure the persistent conformance of their behavior to the shared choreography. Organizations remain free to design and execute their business processes as they see fit, subject to a single *shared constraint*: before executing any *critical action*, every process must query its associated *enforcing monitor* for permission of executing it. The latter does not constraint non-critical portions of the process, but react to queries in one of three possible ways: (i) *grant* the request, if the action conforms to the choreography; (ii) *postpone* its execution, in case now it would violate the choreography, but some future trace of the overall system may still allow the action to be executed conformly; (iii) *deny* the request entirely when no continuation of the system's behavior can produce a conformant execution. This runtime perspective also accommodates processes that are redesigned and redeployed while the federation is already operating, allowing organizations to adapt to unforeseen changes without losing the guarantees provided by the choreography.

The ChoEn tool is grounded in formal methods: the runtime enforcement monitor is automatically synthesized from the input choreography. We establish

two key correctness properties for the enforcers generated by ChoEn, *transparency* and *soundness*. Transparency guarantees that the monitor does not interfere with the execution of business processes as long as their behavior follows the choreography. Soundness ensures that any violating behavior is *amended* by the monitor so that the choreography is ultimately fulfilled.

In the current implementation, ChoEn takes an input choreography expressed as a multiparty session type (MPST)[16,33] or a BPMN choreography diagram [28] and deploys an enforcing smart contract for each monitored process. During the execution phase, then, each process queries its own monitor, which reacts as already described. The ChoEn pipeline is influenced by the idea of automatically generating finite-state automata from local types, composing them onto multiparty session automata (MSAs), and using the latter to monitor the correctness of the running system or amend its actions [16]. The main drawback of such an approach is that by representing states and transitions of the MSA explicitly would require generating an automaton of exponential size with respect to the global type it was derived from. Thus, as a further contribution of this paper, we also present a *compact* representation that generates MSA states and transitions on the fly, thus avoiding such an exponential blow-up.

Two further challenges undertaken by ChoEn deal with the questions: (i) who is *accountable* for the monitor's decisions, and (ii) can the monitor decisions be trusted by all parties? Delegating the monitor to a single organization or third party intermediary would merely relocate the trust problem: the other participants would still have to trust the operator not to tamper with the monitor's state or execution. When designing ChoEn, one of our early architectural decisions was to make it capable of synthesizing *decentralized runtime enforcers* in the form of *smart contracts* that can be deployed on *distributed ledger technologies* (DLTs). ChoEn thus synthesizes enforcers as Solidity smart contracts deployable on a blockchain derived from Ethereum [9]. By sharing the address of the enforcing smart contracts, every organization can directly observe the current state of the enforcer and trust its automated decisions, which are stored immutably, supporting accountability and dispute resolution [38,25].

The remainder of the paper is organized as follows. In Section 2 we recall the needed background. Section 3 describes the enforcing logic and the structure of generated smart contracts. Section 4 documents the architecture of the tool and provides some experimental data used to validate it. Section 5 compares our solution to existing research. Section 6 wraps up and contains some final remarks.

2 Preliminaries

The presented work integrates several research strands, including choreography representation languages, runtime monitoring and enforcement, distributed ledger technologies, and smart contracts.

This section is devoted to introduce them in their essence, for the sake of self-containment, and provide useful references for the interested readers. Even

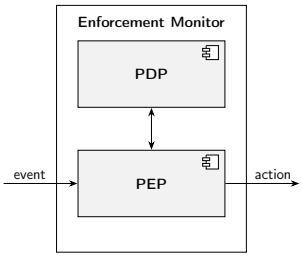


Fig. 1: Zero Trust Architecture [32].

though the ChoEn tool takes as input both BPMN diagrams as well as multiparty session types to represent choreographies, this paper focuses on the latter because details about how to generate smart contracts for monitoring and enforcing BPMN diagrams have been already presented [36].

2.1 Runtime Monitoring and Enforcement

According to NIST, "*a zero trust architecture (ZTA) is an enterprise cybersecurity architecture that is based on zero trust principles and designed to prevent data breaches and limit internal lateral movement.*" [32]. At the core of this architecture lies the *no implicit trust* paradigm, based on the idea of never granting automatic access (e.g. a request originating from a machine connected to the corporate network does not guarantee access to resources). This paradigm is rooted in the well-established principles of *Complete Mediation* (every single access attempt to any resource must be intercepted, mediated, and validated) and *Least Privilege* (users and their devices can be granted only the minimum permissions necessary to complete a specific task, thereby limiting access in terms of both duration and scope). These principles are enabled by:

- *Dynamic policies:* access decisions are not based on static rules but are determined in real-time through a contextual assessment.
- *Continuous monitoring:* the system constantly evaluates the security level, ready to immediately revoke authorizations should risk levels rise.

The architecture proposed by NIST (see Fig. 1) delegates the mediation of every request to the interaction between two components: a Policy Decision Point, tasked with evaluating the context (the Policy Engine or PE) and thus deciding on the most appropriate action (the Policy Administration or PA); and a Policy Enforcement Point (the actual monitor that intercepts every event, asks PDP to evaluate it, and enforces its decision).

A related research area is *Runtime Enforcement* (RE) [3,24,34] which studies formal techniques for ensuring that a running system adheres to a specific policy. The component that monitors the running system is known as an *enforcement monitor*: it takes an untrusted execution as input and modifies it at runtime into an output sequence that complies with the desired policy. An enforcement

monitor is generally required to be sound and transparent: *sound* means the monitor's output must always comply with the specified policy; *transparent* means the monitor must not interfere with a system whose behavior already complies with the specified policy.

It can be observed how the PDP and PEP specified by NIST act together as an enforcement monitor (see Fig. 1). On the other hand, if nothing can be taken for granted, one may even not trust a centralized enforcement monitor. This can be addressed by designing the enforcing monitors to be themselves decentralized and ChoEn adopts this viewpoint, supporting the implementations of ZTAs by providing decentralized enforcers automatically generated from choreographies.

2.2 Multiparty Session Types

Multiparty session types (MPST) are a typing discipline for verifying that message-passing processes in a distributed or concurrent system correctly implement a communication protocol, without communication errors such as unexpected messages or deadlocks. MPST follow a top-down methodology: the design of multiparty protocols begins with a given global type, and each participant's implementation relies on its local type, obtained by the end-point projection of the global type. A global type $\mathcal{G}$ specifies the whole protocol from a birds-eye view — the sequence, choice, and recursion of interactions among all roles — while each local type $\mathcal{T}_p$, obtained via a so-called *end-point projection* (EPP), written $\mathcal{G}\upharpoonright p$ describes the expected communication behavior of a single participant p. A process implementing role p should *conform* to type $\mathcal{T}_p$ along its entire execution. Systems made of processes conformant to their local types guarantee, by compositionality, desirable global properties such as safety, deadlock-freedom, and liveness of interactions. In the following we adopt the Danièlou and Yoshida's [16] equation based syntax for global and local types (see Fig. 2).

We assume input global types are *well-formed*, which greatly simplifies the enforceability of a MPST by means of local monitors. Well-formed global types satisfy the following requirements [16]:

- *unambiguity*: every non-initial state variable $x \in SV$ appears exactly once on a transform left-hand side and once on the right-hand side;
- *unique start*: the initial state variable x_0 appears exactly once on the left-hand side
- *unique end*: the `end` appears at most once;
- *thread correctness*: threads started with Fork are always connected by Join;
- *local choice*: every thread in a Choice share the same *active sender*, i.e. the same process p sending a message, thus making explicit to the receiving processes $p' \in P \setminus \{p\}$ the choice of the branch to be taken;
- *linearity*: no participant can concurrently receive different messages having the same label.

MPSTs have been used in two rather different manners. On one hand, they have been used to define static type-checkers for checking correctness of (white-box) processes expressed in given calculi (e.g. [16,33]). On the other hand, they

$\mathcal{G}$	::=	def G in $\mathbf{x}$	Global type
G	::=	$\mathbf{x} = \mathrm{p} \to \mathrm{p}' : l; \mathbf{x}'$	Labelled messag
	|	$\mathbf{x} = \mathbf{x}' \mid \mathbf{x}''$	Fork
	|	$\mathbf{x} \mid \mathbf{x}' = \mathbf{x}''$	Join
	|	$\mathbf{x} = \mathbf{x}' + \mathbf{x}''$	Choice
	|	$\mathbf{x} + \mathbf{x}' = \mathbf{x}''$	Merge
	|	$\mathbf{x} =$ end	End
$\mathcal{T}$	::=	def T in $\mathbf{x}$	Local type
T	::=	$\mathbf{x} = !\ \mathrm{p}\langle l \rangle.\mathbf{x}'$	Send message
	|	$\mathbf{x} = ?\ \mathrm{p}\langle l \rangle.\mathbf{x}'$	Receive message
	|	$\mathbf{x} = \mathbf{x}'$	Indirection
	|	$\mathbf{x} = \mathbf{x}' \mid \mathbf{x}''$	Fork
	|	$\mathbf{x} \mid \mathbf{x}' = \mathbf{x}''$	Join
	|	$\mathbf{x} = \mathbf{x}' \oplus \mathbf{x}''$	Internal choice
	|	$\mathbf{x} = \mathbf{x}'\ \&\ \mathbf{x}''$	External choice
	|	$\mathbf{x} + \mathbf{x}' = \mathbf{x}''$	Merge
	|	$\mathbf{x} =$ end	End

$\mathbf{x}, \mathbf{x}', \mathbf{x}'' \in SV$

Fig. 2: Grammar for global and local types

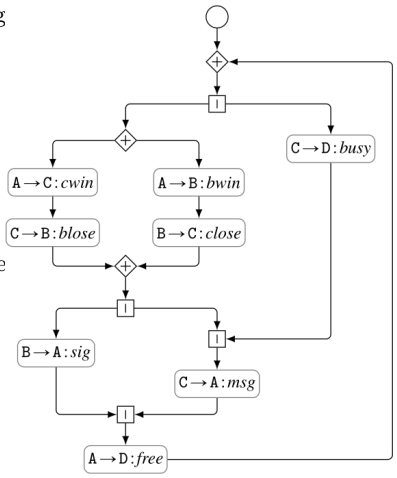


Fig. 3: Running example MPST

have been used as building block to define a *monitorability* relation between (black-box) running processes and global/local-types, along the entire execution of the former (e.g. [21,7,12]). Our work builds upon this second approach, to generate trustful enforcing monitors automatically.

Let us depict inf Fig. 3 a MPST for a lottery-like protocol used as running example and taken from [26]. Its global type is given in Fig. 4b and it is straightforward to check that it is well-formed, since it satisfies the aforementioned properties. Together with it, we report an excerpt of the local types of the involved actors obtained by means of the standard projecting function [16].

2.3 Distributed Ledger Technologies and Smart Contracts

Distributed ledger technology (DLT) [23,5] refers to a class of systems in which a shared, synchronized record of transactions is maintained across a network of independent nodes, without a single centralized authority. Unlike traditional data stores, where a trusted intermediary validates and stores data, a distributed ledger achieves high levels of data integrity among mutually distrustful participants by combining distributed consensus protocols with secure shared transactions based on asymmetric cryptographic protocols. Bitcoin [27] popularized the idea of securely exchanging cryptocurrencies in a decentralized fashion, using a self-governed peer-to-peer architecture. Nevertheless, the significance of DLTs reached a broader applicability with more general-purpose implementations (e.g. Ethereum [9], Hyperledger Fabric [2], ...).

```
G := {
  x0 + x18  = x1
  x1        = x2 | x3
  x2        = x4 + x5
  x4        = Alice->Charlie(cwin); x6
  x5        = Alice->Bob(bwin); x7
  x6        = Charlie->Bob(blose); x8
  x7        = Bob->Charlie(close); x9
  x8 + x9   = x10
  x10       = x11 | x12
  x11       = Bob->Alice(sig); x14
  x12 | x13 = x15
  x3        = Charlie->Dana(busy); x13
  x15       = Charlie->Alice(msg); x16
  x14 | x16 = x17
  x17       = Alice->Dana(free); x18
} in x0;
```

(a) Global type

```
Alice := {
  x0 + x18  = x1
  x1        = x2 | x13
  x2        = x4 (+) x5
  x4        = Alice!cwin(int)->Charlie; x8
  x5        = Alice!bwin(int)->Bob; x9
  x8 + x9   = x10
  x10       = x11 | x12
  x11       = Alice?sig(int)<-Bob; x14
  x12 | x13 = x15
  x15       = Alice?msg(int)<-Charlie; x16
  x14 | x16 = x17
  x17       = Alice!free(int)->Dana; x18
} in x0

Bob := {
  x0 + x18  = x1
  x1        = x2 | x13
  x2        = x6 & x5
  x5        = Bob?bwin(int)<-Alice; x7
  x6        = Bob?blose(int)<-Charlie; x8
  x7        = Bob!close(int)->Charlie; x9
  x8 + x9   = x10
  x10       = x11 | x12
  x11       = Bob!sig(int)->Alice; x14
  x12 | x13 = x16
  x14 | x16 = x18
} in x0

Charlie := {
  ...
}

Dana := {
  ...
}
```

(b) Local types

Fig. 4: The global type and an excerpt of the local types obtained by projecting it for the running example from Fig. 3

Security in DLT systems arises from the distribution and replication of data itself: altering the ledger's history would require simultaneously compromising a majority (or, in some cases, a supermajority) of the network's validating nodes, a task made economically or computationally infeasible by the consensus mechanism in place. This resilience does not eliminate risk entirely but makes it virtually impossible to *repudiate* transactions that have been accepted by validators, making it a secure storage in case of disagreements between the participats. This is especially true in DLTs where there is a large number of validators, e.g. in public blockchains.

It is at this application layer that general-purpose platforms such as Ethereum represent a significant evolution. Indeed, they appealed to the idea of *smart contracts* [37] as a program that can be deployed directly on the blockchain and be executed and inspected *"simultaneously"* by all nodes. To this purpose, Ethereum introduced the idea of a Turing-complete virtual machine (named

Table 1: Enforcer Rules

Name	
Preconditions	Rule
Pass	
$\exists(x = e_p.x') \in \mathcal{T}_p$ $x \in \mathbf{X}_p$	$\dfrac{e_p \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \cdot e_p \quad \beta \quad \mathbf{X}[x \leftarrow_p x'] \setminus_p \mathbb{I}_p(x) \quad \mathbb{I}\setminus_p \{x\}}$
Delay	
$\exists(x = e_p.x') \in \mathcal{T}_p$ $x \notin \mathbf{X}_p$	$\dfrac{e_p \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \quad \beta \uplus_p \{e_p\} \quad \mathbf{X} \quad \mathbb{I}}$
Suppress	
$\nexists(x = e_p.x') \in \mathcal{T}_p$	$\dfrac{e_p \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}$
Resume	
$\exists p \in P.\ \exists(x = e_p.x') \in \mathcal{T}_p$ $e_p \in \beta_p$	$\dfrac{\epsilon \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \cdot e_p \quad \beta \setminus\!\!\setminus_p \{e_p\} \quad \mathbf{X}[x \leftarrow_p x'] \setminus_p \mathbb{I}_p(x) \quad \mathbb{I}\setminus_p \{x\}}$

EVM, for Ethereum Virtual Machine) capable of executing such smart contracts. To complete the picture, special-purpose languages appeared, such as Solidity, a statically-typed, contract-oriented language whose syntax deliberately resembles existing object-oriented procedural languages such as JavaScript and C++. Solidity compiles down to EVM bytecode, allowing contracts to run deterministically and identically on every node in the network, which is essential for maintaining consensus over program execution. This combination of accessibility, compatibility with the EVM standard now adopted by numerous other chains, and an expanding security tooling landscape has made Solidity a reference language for the development of decentralized applications and it motivated also our adoption as target language for the automatic generation of trustful decentralized enforcing monitors presented in this work.

3 Monitoring and Enforcing Choreographies in ChoEn

This section describes in deeper details a original methodology for monitoring and enforcing choreographies expressed as MPSTs. The core idea is defining the enforcer as a labeled transition system (LTS) with rules rewriting a *configuration*. Next, the rules can be straightforwardly implemented as `function`s in Solidity smart contracts. The latter will be deployed onto the selected blockchain, and processes will invoke their enforcing monitor and send requests before attempting critical actions (to this moment, the input MPST can only describe send or receive actions). If process p invokes its smart contract asking permission to execute action a, and the latter returns the requested action a as output symbol,

Table 2: State equivalence

Name	
Preconditions	Rule
Indirection	
$\exists p \in P.\ \exists (x = x') \in \mathcal{T}_p$ $x \in \mathbf{X}_p$	$\dfrac{\epsilon \quad \alpha \quad \beta \quad \mathbf{X} \quad I}{\epsilon \quad \alpha \quad \beta \quad \mathbf{X}[x \leftarrow_p x'] \quad \mathbb{I}[x \leftarrow_p x']}$
Fork	
$\exists p \in P.\ \exists (x = x' \mid x'') \in \mathcal{T}_p$ $x \in \mathbf{X}_p$	$\dfrac{\epsilon \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \quad \beta \quad \mathbf{X}[x \leftarrow_p \{x', x''\}] \quad \mathbb{I}[x \leftarrow_p \{x', x''\}]}$
Join	
$\exists p \in P.\ \exists (x \mid x' = x'') \in \mathcal{T}_p$ $x, x' \in \mathbf{X}_p$	$\dfrac{\epsilon \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \quad \beta \quad \mathbf{X}[\{x, x'\} \leftarrow_p x''] \quad \mathbb{I}[\{x, x'\} \leftarrow_p x'']}$
Internal/External choice	where $\boxplus \in \{\oplus, \&\}$
$\exists p \in P.\ \exists (x = x' \boxplus x'') \in \mathcal{T}_p$ $x \in \mathbf{X}_p$ $\mathbb{I}' = \mathbb{I}[x \leftarrow_p \{x', x''\}]$ $\mathbb{I}'' = \mathbb{I}'[(x', \mathbb{I}_p(x) \cup \{x''\})_p]$ $\mathbb{I}''' = \mathbb{I}'[(x'', \mathbb{I}_p(x) \cup \{x'\})_p]$	$\dfrac{\epsilon \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \quad \beta \quad \mathbf{X}[x \leftarrow_p \{x', x''\}] \quad \mathbb{I}'''}$
Merge-L	
$\exists p \in P.\ \exists (x + x' = x'') \in \mathcal{T}_p$ $x \in \mathbf{X}_p$	$\dfrac{\epsilon \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \quad \beta \quad \mathbf{X}[x \leftarrow_p x''] \quad \mathbb{I}[x \leftarrow_p x'']}$
Merge-R	
$\exists p \in P.\ \exists (x + x' = x'') \in \mathcal{T}_p$ $x' \in \mathbf{X}_p$	$\dfrac{\epsilon \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \quad \beta \quad \mathbf{X}[x' \leftarrow_p x''] \quad \mathbb{I}[x' \leftarrow_p x'']}$
End	
$\exists p \in P.\ \exists (x = \texttt{end}) \in \mathcal{T}_p$ $x \in \mathbf{X}_p$	$\dfrac{\epsilon \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha \quad \beta \quad \mathbf{X}[x \leftarrow_p \emptyset] \quad \mathbb{I} \backslash_p \{s\}}$

it means the enforcer *granted* such request. On the contrary, if the invoked smart contract does not return a as output symbol, it means the enforcer *denied* or *delayed* such request because it does not conform the input MPST at this stage. If some delayed request a' may become applicable, the smart contract *resumes* it from its internal buffer and returns a' as output symbol to p.

3.1 The Enforcing Monitor Logic

Assume a set of participants P and a set of messages M. The alphabet $\Sigma = \{p_1!p_2(m), p_1?p_2(m) : p_1, p_2 \in P, m \in M$ represents the set of enforceable action requests, or *events*. Intuitively, $p_1!p_2(m)$ means p_1 sends a message m to p_2. The complementary action of p_2 receiving m from p_1 is represented as $p_2?p_1(m)$.

Assume a global type $\mathcal{G}$ from the input MPST. Then assume local types $\mathcal{T}_1, \ldots, \mathcal{T}_n$ obtained applying the *projection* functions $\mathcal{G}\upharpoonright p_i$, for each participant $p_i \in P$ (e.g. see [16]). Note that $\mathcal{T}_1, \ldots \mathcal{T}_n$ share the same set of state variables SV appearing in $\mathcal{G}$. The enforcing monitor is based on the operational semantics of MSAs, networks of Communicating Finite-State Machines (CFSMs), each obtained by automatic translation from MPST local types. We omit the formal details of the construction and refer the interested readers to [16] (cf. Sec. 3.3) for the details.

States in each automaton of an MSA are sets of state variables. To avoid ambiguities, we prefer to call *superstates* (for superposed states) any MSA state $\mathbf{X} \subseteq SV$. Since the MSA synthesized from a global type $\mathcal{G}$ can be exponentially bigger than the input type, we chose to encode the enforcer such that it can generate the states of the MSA on-the-fly. To this purpose, we will make use of incompatibility relations $\mathbb{I}_p \subseteq SV \times SV$, for each process $p \in P$. Let us write $\mathbb{I}_p(x)$ to denote the set A such that $a \in A \Leftrightarrow \mathbb{I}_p(x, a)$. In the following we assume that incompatibilities relations in $\mathbb{I}$ are always *closed under symmetry*.

Definition 1. *(Configuration) An enforcer* configuration *is any tuple* $(\alpha, \beta, \mathbf{X}, \mathbb{I})$*, where:* $\alpha \in \Sigma^\star$ *is an output stream;* $\beta = \{\beta_p\}_{p \in P}$ *is a family of multisets* β_p *of monitorable events in* Σ*;* $\{\mathbf{X}_p \subseteq SV\}_{p \in P}$ *denotes a superstate; and* $\mathbb{I} = \{\mathbb{I}_p\}_{p \in P}$ *denotes a family of incompatibility relations.*

A rule r of the enforcing monitor has the following form:

$$\frac{\sigma_p \quad \alpha \quad \beta \quad \mathbf{X} \quad \mathbb{I}}{\alpha' \quad \beta' \quad \mathbf{X}' \quad \mathbb{I}'}$$

Above the line is reported a symbol and a configuration, reflecting the assumptions on the environment before the rule is applied. Below the line is reported a configuration, reflecting the effects of applying the rule on the environment.

Assume $\mathbb{I}$ is a family of incompatibility relations, $s \in SV$ is a state variable and $A \subseteq SV$ a set of state variables . Let us define auxiliary operators *add*, *remove*, and *replace* over $\mathbb{I}$.

Add is written $\mathbb{I}[(s, A)_p]$ and it denotes the family $\mathbb{I}'$ such that: (i) $\forall p' \neq p.\ \mathbb{I}'_{p'} = \mathbb{I}_{p'}$, (ii) $\forall x, y \in SV : x \neq s \wedge y \neq s.\ \mathbb{I}'_p(x, y) \Leftrightarrow \mathbb{I}_p(x, y)$, (iii) $\forall x \in A.\ \mathbb{I}'_p(s, x) \wedge \mathbb{I}'_p(x, s)$. Intuitively, this operation marks the state variables in A as incompatible with state variable s and viceversa, for process p.

Delete is written $\mathbb{I}\backslash_p A$ and it denotes the family $\mathbb{I}'$ such that: (i) $\forall p' \neq p.\ \mathbb{I}'_{p'} = \mathbb{I}_{p'}$, (ii) $\forall x \in SV, a \in A.\ \neg\mathbb{I}'_p(a, x) \wedge \neg\mathbb{I}'_p(x, a)$. Intuitively, this operation removes any incompatibility involving state variables $a \in A$ from the relation $\mathbb{I}_p$.

Finally, replace is written $\mathbb{I}[s \leftarrow_p A]$ and it denotes the family $\mathbb{I}'$ such that: (i) $\forall p' \neq p.\ \mathbb{I}'_{p'} = \mathbb{I}_{p'}$, (ii) $\forall x, y \in SV \setminus \{s\}.\ \mathbb{I}'_p(x, y) \Leftarrow \mathbb{I}_p(x, y)$, and (iii) $\forall x \in$

SV. $\forall a \in A.\ \left[\mathbb{I}'_p(a,x) \vee \mathbb{I}'_p(x,a)\right] \Leftarrow \mathbb{I}_p(s,x)$ Intuitively, this operation replaces any incompatibility (x,s) with incompatibilities (x,a), for $a \in A$, limited to p.

Given a superstate $\mathbf{X}$ and sets $A, B \subseteq SV$, let us write $\mathbf{X}[A \leftarrow_p B]$ to denote the superstate $\mathbf{X}'$ such that: (i) $\forall p' \neq p.\ \mathbf{X}'_{p'} = \mathbf{X}_p$ and (ii) $\mathbf{X}'_p = (\mathbf{X}_p \setminus A) \cup B$. In words, $\mathbf{X}'$ results from superstate $\mathbf{X}$ after replacing state variables from A with those from B, limited to process p. In case A or B are singletons, we may use this notation specifying the item in place of the set, for the sake of conciseness. Let us write $\mathbf{X} \setminus_p A$ to denote the superstate $\mathbf{X}'$ such that: (i) $\forall p' \neq p.\ \mathbf{X}'_{p'} = \mathbf{X}_{p'}$ and (ii) $\mathbf{X}'_p = \mathbf{X}_p \setminus A$.

Let us call *buffer* any mapping β from processes to multisets of events. Let us denote with $\uplus$ and $\setminus\!\!\setminus$ the usual operations of unions and set difference over multisets. Given a buffer β and set $A \subseteq SV$, let us denote by $\beta \uplus_p A$ the buffer β' s.t. $\forall p' \neq p.\ \beta'_{p'} = \beta_{p'}$ and $\beta'_p = \beta_p \uplus A$. Similarly, with $\beta \setminus\!\!\setminus_p A$ we denote the buffer β' s.t. $\forall p' \neq p.\ \beta'_{p'} = \beta_{p'}$ and $\beta'_p = \beta_p \setminus\!\!\setminus A$.

Rules are summarized in Tab. 1. The first group (*Pass*, *Delay*, *Suppress*) are "triggered" on arrival of some input event $e_p \in \Sigma$ generated by process p. They compare e_p with send/receive of the local type $\mathcal{T}_p$ and react accordingly. Rule *Resume* checks whether some buffered event e_p may be applicable given the current state.

Rules reported in Tab. 2, instead, compute on-the-fly the closure of each superstate $\mathbf{X}_p$ modulo the equivalence relation $\equiv_{\mathcal{T}_p}$ defined in [16].

Theorem 1. *Assume a system of processes P and a global type G. Every synthesized enforcer E is* sound *and* transparent *with respect to G.*

3.2 The Generated Solidity Smart Contracts

Fig. 5 describes the structure of the synthesized Solidity smart contracts. The preamble contains enumerations for entities such as `ACTIONS`, `STATES`, `ACTORS`, and `MSG`. The last three depend on the specific MPST given as input. Still in the preamble there are some auxiliary functions organized in libraries, mostly for manipulating arrays of `STATES`, that we omit for the sake of conciseness. Next, there is an abstract contract structure, followed by multiple `contract`s extending it, one for each participant in the choreography.

Each `contract` has a few attributes related to the process $p \in P$ being monitored: `states` encodes p's superstate; `pr` encodes p's incompatibility relation; `delayable` contains a list of events that p should delay using its buffer (cf. rule *Delayed*); `buffer` contains the process' actual buffer; `input` is the list of monitored events as they were generated by p; `output` contains solely the monitored events that made it to the output stream (cf. rules *Pass* and *Resume*). Each contract has two core functions: `check_task` is responsible for testing and applying rules triggered by receiving some input event e_p from the monitored process p (i.e. rules *Pass*, *Delay*, and *Suppress*); `eq_closure` instead tests and applies the remaining rules, that are not triggered by any input event. This explains why only the former receives an input parameter `Event calldata ev`. The contract's constructor preloads the initial state variable in the encoding of

```
pragma solidity >= 0.7.6;
pragma abicoder v2;

enum ACTION { Snd, Rcv }
enum STATE { x0, x1, x2, ... }
enum ACTOR { Alice, Bob, ... }
enum MSG { bwin, cwin, ... }}

struct Event {
  ACTOR initiator;
  ACTION action;
  ACTOR target;
  MSG label;
}

struct IRel {
  mapping(STATE => STATE[]) to_prune;
  mapping(uint256 => bool) pairs;
}

... // libraries States, Events, and Incomp

abstract contract MSABase {
  STATE[] states;
  IRel pr;
  Event[] delayable; Event[] buffer;
  Event[] input; Event[] output;

  constructor() {
    states.push(STATE.x0);
    eq_closure(); load_delayable();
  }

  function eq_closure() public virtual;
  function check_event(ACTOR initiator, ACTION act, ACTOR recipient,
                       MSG label) public virtual;
  function load_delayable() public virtual;
  // ... other utils
}

contract Alice is MSABase { ... }
contract Bob is MSABase { ... }
contract Charlie is MSABase { ... }
contract Dana is MSABase { ... }
```

Fig. 5: Skeleton of the synthesized decentralized Solidity enforcer.

```
function check_event(ACTOR a, ACTION act, ACTOR b, MSG label) public {
  Event memory ev = Event(a, act, b, label);
  input.push(ev);

  // Send : x4 = Alice!cwin(int)->Charlie; x8
  if (Events.eq(ev, ACTION.Snd, ACTOR.Alice, ACTOR.Charlie, MSG.cwin) &&
      apply_pass(states, pr, STATE.x4, task, STATE.x8)) {
        output.push(ev); eq_closure(); return;
  }

  // Receive : x11 = Alice?sig(int)<-Bob; x14
  if (Events.eq(ev, ACTION.Rcv, ACTOR.Alice, ACTOR.Bob, MSG.sig) &&
      apply_pass(states, pr, STATE.x11, task, STATE.x14)) {
        output.push(ev); eq_closure(); return;
  }
  // ... more rules for Send and Receive

  // add to buffer (delay) or skip (suppress)
  if (Events.contains(delayable, ev)) { buffer.push(ev); }
}

function eq_closure() internal override {
  bool dirty = false;

  do {
    dirty = false;

    if (apply_merge(states, STATE.x0, STATE.x18, STATE.X1)) {
      // merge : x0 + x18 = x1
      dirty = true;
    } else if (apply_fork(states, pr, STATE.x1, STATE.x2, STATE.x13) {
      // fork: x1 = x2 | x13
      dirty = true;
    } else if (apply_choice(states, STATE.x2, STATE.x4, STATE.x5) {
      // internal/external choice: x2 = x4 (+) x5
      dirty = true;
    } else if (apply_join(states, pr, STATE.x12, STATE.x13, STATE.x15) {
      // join: x12 | x13 = x15
      dirty = true;
    } else if ... // other rules

  } while (dirty);
}
```

Fig. 6: `Alice`'s function `check_event` contains the encoding of *Pass*, *Delay*, and *Suppress*, while function `eq_closure` encodes the remaining rules

the current superstate, then invokes `eq_closure` to "cross" all the state variables that are equivalent to the initial one based on $\equiv_{\mathcal{T}_p}$, and finally loads in array `delayable` all the events that the monitored process may delay using the buffer.

In the current implementation, the rules are implemented as "code-patterns" inside the functions `check_event` and `eq_closure`. Let us display them in relation to the running example (cf. Fig. 6).

Inside `check_event`, the preconditions of the rules are encoded as `if (...)` conditions. In case the test is positive, the function `apply_pass` is invoked and the current triggering event `ev` is enqueued in the output of the array `output`. In case of an invocation of `apply_pass`, state variables in the superstate $\mathbf{X}$ change to reflect the just observed event. Function `eq_closure()` is invoked to cross all the state variables that are equivalent to the ones contained in $\mathbf{X}_p$, for each p, based on the already mentioned equivalence relation $\equiv_{\mathcal{T}_p}$. In case no *Pass* rule applies for any send/receive transform in the local type, the current event `ev` is appended to the contract's `buffer` array if `ev` is one of the events to be delayed (cf. rule *Delay*), otherwise it is just ignored (cf. rule *Suppress*).

4 Architecture and Validation

The ChoEn tool divides the interaction with the user in two successive stages, called *Compile-Time* (CT) and *Run-Time* (RT), as depicted in Figure 7.

During CT, a choreography (either MPST or BPMN diagram) is passed as input using one of the available interfaces (CLI or REST API). Since the architectural components for processing BPMN diagrams have been already documented [36], in this work we document those dealing with MPSTs.

First, the global type is extracted from the MPST and projected onto multiple local types (through *ProjLT*). The graph describing the local type is passed to the *Smart Contract Generator* that is responsible for the automatic compilation of three Solidity smart contracts as described in Sec. 3. A component named *Smart-Contract Manager* is in charge of deploying the generated smart contracts (one monitor for each local type) onto a chosen Blockchain and store their addresses and ABIs, thus allowing future interactions and integration of third-party applications along the RT phase. This workflow is automatic and executed once for every choreography passed as MPST.

During RT, instead, the monitored processes interact with the *Enforcer Proxy* (through the CLI or the REST API) providing one or more event symbols as input. The contract invoked by the process react and updates its internal configuration to reflect the rules that have been triggered by the provided event symbol, possibly returning some output event symbols resulting from enforcing the input events just provided.

In Tab. 3 we report some data collected during Compile-Time and Run-Time phases of several test cases, included the running example (first row). Each test case correspond to a choreography defined using MPST global types, taken from research papers, converting BPMN diagrams onto MPSTs if possible, or modeling some real-world industrial scenarios. In order to simulate realistic scenarios

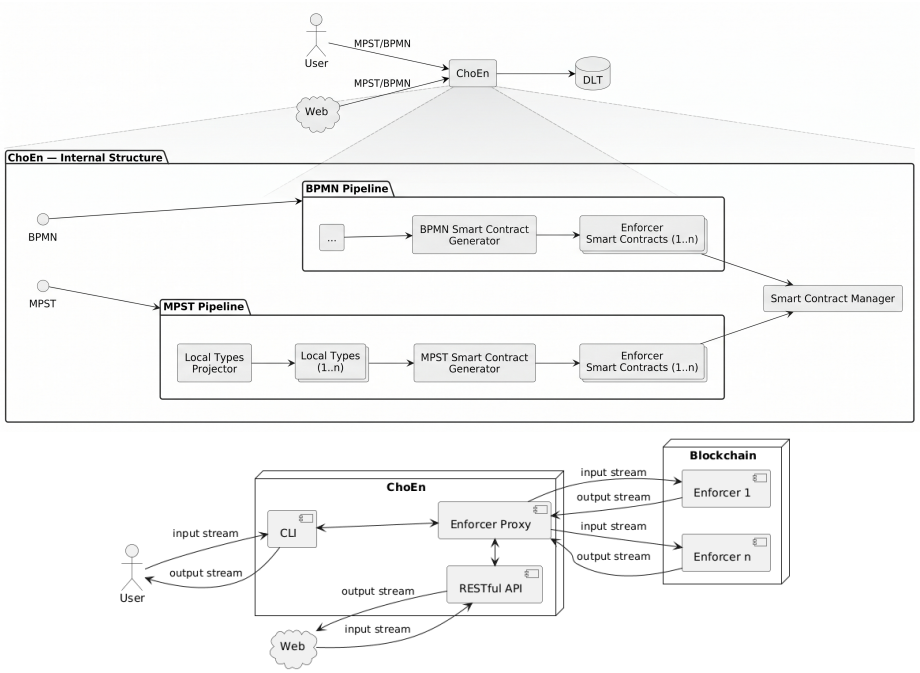


Fig. 7: Compile-time (above) and run-time (below) pipelines

where processes behave correctly "most of the times", we encoded a simulation logic where processes behave as events emitter using a special weighted random function: 80% of the times it emits choreography messages in the expected order, and 20% of the times it emits choreography messages out of order. The scenarios

Table 3: Summary of data collected on our test-bench.
(Legend: $P \rightarrow \#participants, SV \rightarrow \#state\ variables, M \rightarrow \#messages$)

	Compile-time				Runtime		
	MPST			Solidity	Sepolia Testnet		
Choreography	*P*	*SV*	*M*	*Init Size (bytes)*	*Exc. msg.*	*GAS*	*Execution (sec)*
Lottery	4	19	8	295,612	12	1,007,973	31.40
Alternating bit	2	15	4	155,464	14	1,050,003	24.97
Patient-doctor	3	7	6	136,792	12	832,086	28.15
Railway maintenance	6	26	13	430,472	26	851,408	24.65
Incident management	5	23	8	307,128	20	643,627	24.87
Coffee machine	2	10	5	140,988	10	685,455	30.67
			Average	*244,409.3*		*845,092*	*27.45*

have been first simulated locally using the Ganache suite by Truffle. Next, the ChoEn tool has been connected to the Sepolia Ethereum testnet, in order to test our tool under more realistic assumptions.

The conducted experiments confirm that the tool is usable in realistic scenarios. The added execution time is in the order of tenths of seconds for an entire choreography, thus compatible with usual latencies of services provided over the Web. The actual cost of the enforcement service in ETH (and thus in fiat currencies such as EUR and USD) may vary considerably, depending on the chosen DLT and on the GAS cost imposed by the DLT.

5 Related Work

Van den Heuvel et al. [21] propose a blackbox monitoring approach, treating monitored components as abstract labelled transition systems, similar to our approach. Their technique, however, synthesizes monitors directly from global types, and correspondingly requires the definition of *relative types* — pairwise projections that retain cross-role dependency information — rather than relying on the sole local types as we do, thus our methodology is simpler in its foundations. Moreover, their work is limited to monitoring and does not address enforcement of protocol conformance, nor does it consider using DLTs as a backbone for their monitors. Earlier, Chen et al. [12] and Bocchi et al. [7] developed a theory of distributed monitors for multiparty session types, attaching a monitor derived from each participant's local type to that participant's endpoint process. Their formal model, however, requires each monitored process to be given as a term of a specific π-calculus, so that the monitor's reduction semantics can be defined by structural induction on the process; consequently, their approach is not, strictly speaking, black-box and rely upon knowing the processes to be monitored, an assumption that we do not need. In addition, both works do not address the problem of runtime enforcement and do not make use of DLTs.

Scalas and Yoshida [33] present MPSTK, a prototype tool implementing a static type-checking algorithm for processes expressed as terms in a suitable π-calculus. Their contribution, however, is focused on properties derived from static typing and rely on knowing the structure of the processes to be typed. Thus, it does not address the runtime monitoring or enforcement of process conformance that is the focus of the present work.

Gudemann et al. [20] introduce automatic procedures for synthesizing controllers that monitor and enforce correct execution of processes with respect to choreographies given as Conversation Protocols [17]. Later, they introduce a tool, VerChor [19] taking as input choreographies specified using an intermediate language they define CIF (Choreography Intermediate Format) able to specify conversation protocols and BPMN choreography diagrams, and possibly other choreography specification languages. They present a procedure for generating controllers that, by wrapping the monitored processes, achieve enforceability of choreographies that are non-realizable. In their scenario runtime enforcement of running processes is an option in case the choreography is not realizable (and

thus correct-by-construction processes cannot be synthesized). In our scenario, on the contrary, we assume that behaviors of running processes should always be monitored and enforced for security purpose. Also, our assumption is that the MPST provided as high-level specification of the system is well-formed and thus realizable. Finally, their work did not consider the problem of synthesizing decentralized enforcing monitors using smart contracts to be executed on DLTs.

Corradini et al. [14,15] as well as Weber et al. [25,38,31] experimented multiple approaches to generate decentralized monitors as smart contracts executable on blockchains. They all take as input BPMN choreography diagrams. One work tested the usage of Bitcoin smart contracts [31] while the others compile enforcing monitors as Solidity smart contracts targeting Ethereum-like DLTs. Their approaches differs because they rely more on a model-driven approach to generation of smart contract. In model-driven approaches, the researchers present rules for generating smart contract code out of syntactic structures appearing in the BPMN choreography diagrams. In particular, they do not formally document the rules of implemented by their enforcers. Consequently, they do not tackle the problem of proving the soundness and transparency of such enforcers, nor the correctness of the generated smart contracts.

Recent work has explored the decentralization of individual functions within Zero Trust Architectures (ZTAs). Under the NIST decomposition, however, a fully decentralized PDP would require decentralizing both the Policy Engine (PE) and the Policy Administrator (PA). However, Hu et al. [22] describe a blockchain-based decentralized ABAC PDP, where only the PE is decentralized and their model does not explicitly use past request history as an input to access decisions. Bicer et al. [6] replicate the PE across multiple off-chain instances, while a PA coordinates their decisions. Their blockchain records access requests and decisions, and the PE can use request history for behavioral checks. Alsulami et al. [1] combine AI-based trust scoring with smart-contract-based PA and blockchain audit logging. Thus, all these works decentralize or blockchain-enable only selected PDP functions; none demonstrates full decentralization of both the PE and PA, as done by ChoEn instead.

Finally, this paper extends a preliminary presentation of the ChoEn tool taking as input only BPMN choreography diagrams [36]. The ChoEn tool itself was devised as a means to formalize an older, model-driven, approach to enforcement of BPMN choreography diagrams applied in some construction projects [35,13]. The first version of ChoEn could take as input only *regular-enough* BPMN choreography diagrams. Such assumption ruled out less regular but common choreography structures, such as the lottery protocol used as running example.

6 Conclusions

This work stems from a twofold lack of trust commonly observed in inter-organizational cooperating contexts. First, individual organizations cannot be fully trusted to adhere to the terms of an agreement. Second, should a controller be appointed, the participating organizations may not trust its decisions,

just as they do not trust one another. We are, therefore, dealing with a genuine zero-trust scenario. Solutions proposed for such scenarios typically rely on a centralized monitor or on partially decentralized monitors. In this way, they address the first type of mistrust but they fall prey to the second. To fully address both types of mistrust, we present a tool named ChoEn.

Through it, smart contracts are automatically generated and deployed on a distributed ledger. Running processes can then query the contracts to request authorization for critical actions, such as sending and receiving messages. Based on the choreography and the execution history, the enforcers may grant, delay, or suppress such actions. ChoEn currently accepts choreographies specified either as multiparty session types or as BPMN choreography diagrams. The tool REST API allows integration with third-party applications that may benefit from our approach. The enforcers generated by ChoEn record enforcement-related decisions on a distributed ledger, where it can be inspected by the participants and cannot be repudiated by the enforcing monitors. Since the enforcers are generated automatically from the choreography, the third-party applications are exempt from implementing the underlying smart contracts or verifying their correctness. We validated the tool by simulating several scenarios, verifying that the enforcers apply the prescribed rules in the expected order. The collected data also indicate that enforcement can be performed with a limited impact on the execution of the monitored processes. The cost of providing the enforcement service through Solidity smart contracts on Ethereum-like blockchains, however, depends substantially on the exchange rate between the cryptocurrency used to pay transaction fees (e.g., ETH) and traditional fiat currencies.

A current threat to the applicability of our approach is the lack of immediate deterministic finality for just-validated blocks on some DLT platforms. On Ethereum —the current target platform of ChoEn— for instance, an approved event may be recorded in a block that is subsequently replaced by a competing branch before the transaction reaches the required level of finality. This may cause the state observed by the on-chain enforcer to diverge from the state actually reached by the processes it controls, potentially affecting subsequent enforcement decisions. DLTs such as Algorand [11], on the other hand, provide stronger finality guarantees under the assumptions of their consensus protocol, making the probability of a finalized block being reverted negligible [18,10]. We therefore plan to extend ChoEn by allowing users to choose the target language of the generated smart contracts, adding TEAL or PyTeal alongside Solidity, which is currently the only supported option. This will allow the enforcement mechanism to be deployed on DLT platforms with different consensus protocols and finality guarantees, and will enable us to investigate how these properties affect the applicability of decentralized choreography enforcement.

Finally, we plan to integrate ChoEn with asynchronous publish/subscribe communication patterns (e.g. REDIS, MQTT, ...), to accept choreographies expressed in the VeriChor CIF format VeriChor [19], and to integrate techniques for automatically correcting non-realizable choreographies so that they can still be controlled at runtime.

References

1. Alsulami, F.N.: Adaptive-AI-ZeroTrust-Chain: Blockchain-backed dynamic zero-trust enforcement via artificial intelligence. Journal of King Saud University Computer and Information Sciences **38**(262) (2026). https://doi.org/10.1007/s44443-026-00622-9, https://doi.org/10.1007/s44443-026-00622-9
2. Androulaki, E., Barger, A., Bortnikov, V., Cachin, C., Christidis, K., De Caro, A., Enyeart, D., Ferris, C., Laventman, G., Manevich, Y., Muralidharan, S., Murthy, C., Nguyen, B., Sethi, M., Singh, G., Smith, K., Sorniotti, A., Stathakopoulou, C., Vukolić, M., Weed Cocco, S., Yellick, J.: Hyperledger fabric: A distributed operating system for permissioned blockchains. In: Proceedings of the 13th EuroSys Conference. pp. 1–15. ACM (2018). https://doi.org/10.1145/3190508.3190538
3. Bartocci, E., Falcone, Y., Francalanza, A., Reger, G.: Introduction to runtime verification. In: Bartocci, E., Falcone, Y. (eds.) Lectures on Runtime Verification: Introductory and Advanced Topics, Lecture Notes in Computer Science, vol. 10457, pp. 1–33. Springer (2018). https://doi.org/10.1007/978-3-319-75632-5_1
4. Basu, S., Bultan, T., Ouederni, M.: Deciding choreography realizability. ACM Sigplan Notices **47**(1), 191–202 (2012). https://doi.org/10.1145/2103656.2103680
5. Belotti, M., Bozic, N., Pujolle, G., Secci, S.: A vademecum on blockchain technologies: When, which, and how. IEEE Communications Surveys & Tutorials **21**(4), 3796–3838 (2019). https://doi.org/10.1109/COMST.2019.2928178
6. Bicer, C., Murturi, I., Donta, P.K., Dustdar, S.: Blockchain-based zero trust on the edge. In: 2023 International Conference on Computational Science and Computational Intelligence (CSCI). pp. 1006–1013. IEEE (2023). https://doi.org/10.1109/CSCI62032.2023.00167, https://doi.org/10.1109/CSCI62032.2023.00167
7. Bocchi, L., Chen, T.C., Demangeon, R., Honda, K., Yoshida, N.: Monitoring networks through multiparty session types. In: International Conference on Formal Methods for Open Object-Based Distributed Systems. pp. 50–65. Springer (2013)
8. Bravetti, M., Zavattaro, G.: Foundational aspects of contract compliance and choreography conformance. Tech. Rep. MSR-TR-2008-61, Microsoft Research (2008), https://www.microsoft.com/en-us/research/wp-content/uploads/2008/05/tr-2008-61.pdf
9. Buterin, V.: Ethereum: A next-generation smart contract and decentralized application platform. Ethereum White Paper (2014), https://ethereum.org/whitepaper/
10. Chen, J., Gorbunov, S., Micali, S., Vlachos, G.: ALGorand agreement: Super fast and partition resilient byzantine agreement. Cryptology ePrint Archive, Paper 2018/377 (2018), https://eprint.iacr.org/2018/377
11. Chen, J., Micali, S.: Algorand: A secure and efficient distributed ledger. Theoretical Computer Science **777**, 155–183 (2019). https://doi.org/10.1016/j.tcs.2019.02.001
12. Chen, T.C., Bocchi, L., Deniélou, P.M., Honda, K., Yoshida, N.: Asynchronous distributed monitoring for multiparty session enforcement. In: International Symposium on Trustworthy Global Computing. pp. 25–45. Springer (2011)
13. Corneli, A., Naticchia, B., Spegni, F., Spalazzi, L.: Combining blockchain and BPMN choreographies for construction management. In: EC3 Conference 2021. vol. 2, pp. 34–41. European Council on Computing in Construction (2021). https://doi.org/10.35490/EC3.2021.204
14. Corradini, F., Marcelletti, A., Morichetta, A., Polini, A., Re, B., Tiezzi, F.: Engineering trustable choreography-based systems using blockchain. In: Proceedings of the 35th Annual ACM Symposium on Applied Computing. pp. 1470–1479 (2020). https://doi.org/10.1145/3341105.3373988

15. Corradini, F., Marcelletti, A., Morichetta, A., Polini, A., Re, B., Scala, E., Tiezzi, F.: Model-driven engineering for multi-party business processes on multiple blockchains. Blockchain: Research and Applications **2**(3), 100018 (2021). https://doi.org/10.1016/j.bcra.2021.100018
16. Deniélou, P.M., Yoshida, N.: Multiparty session types meet communicating automata. In: European Symposium on Programming. pp. 194–213. Springer (2012)
17. Fu, X., Bultan, T., Su, J.: Conversation protocols: A formalism for specification and verification of reactive electronic services. Theoretical Computer Science **328**(1–2), 19–37 (2004). https://doi.org/10.1016/j.tcs.2004.07.004
18. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N.: Algorand: Scaling byzantine agreements for cryptocurrencies. In: Proceedings of the 26th ACM Symposium on Operating Systems Principles. pp. 51–68. SOSP '17, ACM, New York, NY, USA (2017). https://doi.org/10.1145/3132747.3132757
19. Güdemann, M., Poizat, P., Salaün, G., Ye, L.: VerChor: A framework for the design and verification of choreographies. IEEE Transactions on Services Computing **9**(4), 647–660 (2015). https://doi.org/10.1109/TSC.2015.2413401
20. Güdemann, M., Salaün, G., Ouederni, M.: Counterexample guided synthesis of monitors for realizability enforcement. In: Automated Technology for Verification and Analysis. Lecture Notes in Computer Science, vol. 7561, pp. 238–253. Springer, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-33386-6_20
21. van den Heuvel, B., Pérez, J.A., Dobre, R.A.: Monitoring blackbox implementations of multiparty session protocols. In: International Conference on Runtime Verification. pp. 66–85. Springer (2023)
22. Hu, Q., Correia, M., Jiang, T.: An efficient blockchain for decentralized abac policy decision point. Future Generation Computer Systems **166**, 107732 (2025). https://doi.org/10.1016/j.future.2025.107732, https://doi.org/10.1016/j.future.2025.107732
23. Kannengiesser, N., Lins, S., Dehling, T., Sunyaev, A.: Trade-offs between distributed ledger technology characteristics. ACM Computing Surveys **53**(2), 1–37 (2020). https://doi.org/10.1145/3379463
24. Khoury, R., Tawbi, N.: Which security policies are enforceable by runtime monitors? a survey. Computer Science Review **6**(1), 27–45 (2012). https://doi.org/10.1016/j.cosrev.2012.01.001
25. Ladleif, J., Weske, M., Weber, I.: Modeling and enforcing blockchain-based choreographies. In: Hildebrandt, T., van Dongen, B.F., Röglinger, M., Mendling, J. (eds.) Business Process Management – 17th International Conference, BPM 2019, Proceedings. Lecture Notes in Computer Science, vol. 11675, pp. 69–85. Springer (2019). https://doi.org/10.1007/978-3-030-26619-6_7
26. Lange, J., Tuosto, E., Yoshida, N.: From communicating machines to graphical choreographies. In: Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. pp. 221–232 (2015)
27. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system (2008), https://bitcoin.org
28. OMG: Business Process Model and Notation (BPMN), Version 2.0. Standard, Object Management Group (2011), https://www.omg.org/spec/BPMN/2.0
29. Paulson, L.D.: Choreographing web services. Computer **35**(11), 23–25 (Nov 2002). https://doi.org/10.1109/MC.2002.1046969
30. Peltz, C.: Web services orchestration and choreography. Computer **36**(10), 46–52 (Oct 2003). https://doi.org/10.1109/MC.2003.1236471

31. Prybila, C., Schulte, S., Hochreiner, C., Weber, I.: Runtime verification for business processes utilizing the bitcoin blockchain. Future Generation Computer Systems **107**, 816–831 (2020). https://doi.org/10.1016/j.future.2017.08.024
32. Rose, S., Borchert, O., Mitchell, S., Connelly, S.: Zero trust architecture (2020-08-10 04:08:00 2020). https://doi.org/https://doi.org/10.6028/NIST.SP.800-207, https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=930420
33. Scalas, A., Yoshida, N.: Less is more: multiparty session types revisited. Proceedings of the ACM on Programming Languages **3**(POPL), 1–29 (2019)
34. Schneider, F.B.: Enforceable security policies. ACM Transactions on Information and System Security **3**(1), 30–50 (2000). https://doi.org/10.1145/353323.353382
35. Spalazzi, L., Spegni, F., Corneli, A., Naticchia, B.: Blockchain based choreographies: The construction industry case study. Concurrency and Computation: Practice and Experience **35**(16), e6740 (2023)
36. Spegni, F., Fratini, L., Pirani, M., Spalazzi, L.: Choen: A smart contract based choreography enforcer. In: 2023 IEEE international conference on pervasive computing and communications workshops and other affiliated events (PERCOM workshops). pp. 86–91. IEEE (2023)
37. Szabo, N.: Smart contracts. Web manuscript (1994), https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart.contracts.html, accessed: 2026-09-23
38. Weber, I., Xu, X., Riveret, R., Governatori, G., Ponomarev, A., Mendling, J.: Untrusted business process monitoring and execution using blockchain. In: International conference on business process management. pp. 329–347. Springer (2016)

A Formal Proofs

Let us provide here the formal details required to prove soundness and transparency of the generated enforcing monitors.

Given a set of processes P, let us represent by P also the parallel composition of its processes $p_1 \parallel \ldots \parallel p_n$, for $p_i \in P$.

Recalling the definition of MSA from [16], there exists a translation from local types $\mathcal{T}$ to a Communicating Finite-State Machine (CFSM) defined as $\mathcal{A}(\mathcal{T}) = (Q, C, q_0, \mathbf{A}, \delta)$. Q contains states that are sets of state variables appearing in $\mathcal{T}$[3]. q_0 is the initial state variable appearing in $\mathcal{T}$. C is the set of channels (one for each pair of participants p, q appearing in $\mathcal{T}$). $\mathbf{A} = \{l_1, l_2, \ldots\}$ is the set of message labels exchanged in $\mathcal{T}$. $\delta \subseteq Q \times \Lambda \times Q$ is the transition relation induced by transforms appearing in local type $\mathcal{T}$ labeled with $pq!l$ (if it was induced by a sending transform from p to q of a message labeled l) or $qp?l$ (if it was induced by a receiving transform of a message l sent from q and received by p). A global type $\mathcal{G}$ thus induces a MSA $(\mathcal{A}(\mathcal{G}\upharpoonright p))_{p \in \mathcal{G}}$, viz. the composition of the aforementioned CFSMs.

In the following, when we assume that system P is correct we thus assume it is trace-equivalent to a correct MSA $\hat{P}$, and when we refer to P's state we denote the state of $\hat{P}$ which is equivalent to the (unknown) actual state of P.

Let us denote with $p : \mathcal{T}$ the fact that p is trace-equivalent to some CFSM $\hat{p}$ generated from $\mathcal{T}$. Formally:

$$p : \mathcal{T} \Leftrightarrow \exists \hat{p}.\ p \simeq \hat{p} \wedge \hat{p} = \mathcal{A}(\mathcal{T})$$

Similarly, $P : \mathcal{G}$ in our setting denotes $\exists \hat{P}.\ P \simeq \hat{P} \wedge \hat{P} = \{\mathcal{A}(\mathcal{G}\upharpoonright p)\}_{p \in \mathcal{G}}$.

Let us consider any system P as a *generator* of a language $\mathcal{L}_P \subseteq \Sigma^\star$, where Σ is the events dictionary (cf. Sec. 3.1). Let us denote by $\mathcal{L}_\mathcal{G} \subseteq \Sigma^\star$ the language that: (i) is generated by any CSFM $\hat{P}$ provided that $\hat{P} = \{\mathcal{A}(\mathcal{G}\upharpoonright p)\}_{p \in \mathcal{G}}$, and (ii) is *closed under the finite prefixes*. From any system P and global type $\mathcal{G}$, let us denote by $E(P, \mathcal{G})$ the generated enforcer. Its initial configuration is $\iota = (\epsilon, \beta_0, S_0, \mathbb{I}_0)$, where: ϵ denotes the empty output stream; $\beta_0 = \{\emptyset\}_{p \in P}$ denotes the family of empty buffers for every p; $S_0 = \{\{x_0\}\}_{p \in P}$ maps every process p to the superstate $\{x_0\}$; $\mathbb{I}_0 = \{\emptyset\}_{p \in P}$ is the family of empty incompatibility relations, for every process. Given any configuration $c = (\alpha, \beta, S, \mathbb{I})$, let us write $output(c)$ to denote component α, i.e. its output stream. Also, let us write $svars(c)$ to denote S. Let us denote by $c \xrightarrow{\sigma} c'$ the fact that configuration c' is reached from c by applying one of the rules triggered by input event σ (i.e. either *Pass*, *Delay*, or *Suppress*). Let us write $c \rightarrow_\epsilon c'$ the fact that configuration c' is reached from c by applying any of the remaining rules. Let us write $c \overset{\sigma}{\rightsquigarrow} c'$ any sequence $c \xrightarrow{\sigma} c_1 \rightarrow_\epsilon \ldots \rightarrow_\epsilon c_n$, where (i) $c' = c_n$ and (ii) the sub-sequence $c_1 \rightarrow_\epsilon \ldots \rightarrow_\epsilon c_n$ is maximal, i.e. no more rules can be applied to further change configuration c_n without observing a new event. Using the terminology introduced by [16],

[3] Note that states $q \in Q$ are different from our notion of superposed states, or superstates, because each state contains only state-variables that are mutually compatible among themselves

configurations $c_1, \ldots, c_n$ are all equivalent with respect to relation $\equiv_{\mathcal{T}}$, i.e. $c_1 \equiv_{\mathcal{T}} \ldots \equiv_{\mathcal{T}} c_n$ (cf. Sec. 3.3 in [16]). By extension, let us write $c \overset{\alpha}{\rightsquigarrow} c'$ to denote $c \overset{\sigma_1}{\rightsquigarrow} c_1 \overset{\sigma_2}{\rightsquigarrow} \ldots \overset{\sigma_n}{\rightsquigarrow} c_n$, whenever $\alpha = \sigma_1 \cdot \ldots \cdot \sigma_n$ and $c_n = c'$.

Since the enforcer $E(P, \mathcal{G})$ acts as a protective shell around the system of processes P, let us thus define its as follows:

$$\mathcal{L}_{E(P,\mathcal{G})} = \{\alpha : \iota \overset{\alpha'}{\rightsquigarrow} c, \alpha' \in p, \alpha = \mathit{output}(c)\}$$

where relation $\overset{\sigma}{\rightsquigarrow}$ is induced by the rules of enforcer $E(P, \mathcal{G})$.

Clearly, by knowing that a given system P conforms to global type $\mathcal{G}$, we have that $\mathcal{L}_P \subseteq \mathcal{L}_{\mathcal{G}}$. Since we assume processes are black boxes and we don't have access to its structure, we don't know whether $P : \mathcal{G}$. An enforcer is thus usually required to satisfy the following two properties.

Lemma 1. *Let us assume any system P and any global type $\mathcal{G}$. Then:*

$$\mathcal{L}_{E(P,\mathcal{G})} = \mathcal{L}_{\mathcal{A}(\mathcal{G})}$$

Proof. Direction ($\Leftarrow$). Take any $\alpha = \alpha_1 \ldots \alpha_n \in \mathcal{L}_{\mathcal{A}(\mathcal{G})}$. Let us show $\alpha \in \mathcal{L}_{E(P,\mathcal{G})}$. Every automaton $\mathcal{A}(\mathcal{G}\upharpoonright p)$ starts at state $\mathbf{X}_{p,0} = [x_0]_{\equiv_{\mathcal{T}_p}}$. The enforcer $E(P, \mathcal{G})$ instead starts at $\{X_{p,0}\}_{p \in P}$ in c_0 and, for some $i \geq 0$, there exists c_i such that $c_0 \rightarrow_\epsilon c_1 \rightarrow_\epsilon \ldots \rightarrow_\epsilon c_i$ using rules in Tab. 2. By definition of $\rightarrow_\epsilon$, we have that $c_i \equiv_{\mathcal{T}} c_0$.

Let us prove this by induction on the length of α. For a string $\alpha = \epsilon$ it is trivially true: $\epsilon \in E(P, \mathcal{G})$. In the inductive case, assume (i) $\alpha \in \mathcal{L}_{E(P,\mathcal{G})}$, (ii) there is a MSA state $\{\mathbf{X}_{p,n}\}_{p \in P}$ recognizing α, (iii) $\alpha \in \mathcal{L}_{E(P,\mathcal{G})}$, and (iv) there exists a configuration c_n s.t. $c_0 \overset{\alpha}{\rightsquigarrow} c_n$. Let us now focus on some $\alpha' = \alpha \cdot e_p$ for some $e_p \in \Sigma$, i.e. a new event symbol has been issued by some process $p \in P$ and a transition $\delta_p(\mathbf{X}_{p,n}, e_p, \mathbf{X}_{p,n+1})$. This yields, by definition of MAS [16], that (i) there exists a transform `x = p!m->q ; x'` (resp. `x = p?m<-q ; x'`) in $\mathcal{T}_p$, (ii) `x` $\in \mathbf{X}_{p,n}$, (iii) `x'` $\in \mathbf{X}_{p,n+1}$, and $e_p =$ `p!m->q` (resp. $e_p =$ `p?m<-q`). Those conditions are exactly those that enable rule *Pass* in $E(P, \mathcal{G})$, thus ensuring that e_p is appended to the output stream. Formally, there exists c_{n+1} such that $c_n \overset{e_p}{\rightsquigarrow} c_{n+1}$ and $\mathit{output}(c_{n+1}) = \alpha_1 \ldots \alpha_n e_p = \alpha'$.

Direction ($\Rightarrow$). Take any $\alpha = \alpha_1 \ldots \alpha_n \in \mathcal{L}_{E(P,\mathcal{G})}$. Let us show $\alpha \in \mathcal{L}_{\mathcal{A}(\mathcal{G})}$. Let us prove this by induction on the length of α. The base case trivially holds, i.e. $\epsilon \in \mathcal{L}_{\mathcal{A}(\mathcal{G})}$, since the latter is by definition closed under prefixes. In the inductive case, we may assume some $\alpha = \alpha_1 \ldots \alpha_n$ s.t. $\alpha \in \mathcal{L}_{E(P,\mathcal{G})}$ and $\alpha \in \mathcal{L}_{\mathcal{A}(\mathcal{G})}$. First of all, let us observe that α may not be the original input of events as received by the enforcer $E(P, \mathcal{G})$, but only its output. This means (i) there exists some (equally long or longer) $\hat{\alpha}$ and some configuration c_n in enforcer $E(P, \mathcal{G})$ such that $c_0 \overset{\hat{\alpha}}{\rightsquigarrow} c_n$, and (ii) there exist some MSA state $\{\mathbf{X}_{p,n}\}_{p \in P}$ such that for all $i \in [1, n]$, $\mathbf{X}_{i-1,p_i} \xrightarrow{\alpha_i}_{\delta_{p_i}} \mathbf{X}_{i,p_i}$, where $p_1 \ldots p_n$ is the sequence of processes issuing event symbols $\alpha_1 \ldots \alpha_n$, respectively. Note that only "accepted" events α_i in α may induce a transition in the MSA. Assume any new event symbol e_p issued by some process $p \in P$ (denoting either a send or a receive action); let us name

$\alpha' = \alpha \cdot e_p$ the output stream in this inductive case. Only two rules may allow symbol e_p in the output component of the enforcer: *Pass* or *Resume*. Thus, there exists c_{n+1} such that $c_n \xrightarrow{e_p} c_{n+1}$ and $output(c_{n+1}) = \alpha'$. Let us reason by cases.

Case 1: rule *Pass* has been applied. By the rule precondition, we have that (i) $\exists(x = e_p.x') \in \mathcal{T}_p$ and (ii) $x \in \mathbf{X}_{p,n}$. By the effects of the same rule, (iii) $\mathbf{X}_{p,n+1}$ is obtained from $\mathbf{X}_{p,n}$ by replacing state variable x with x', and removing all state variables x'' marked as alternative to x by some previous application of rule *Internal/External Choice* and carried on by other rules in Table. 2 (i.e. any $x'' \in \mathbb{I}_p(x)$). Observations (i), (ii), and (iii) implies that the transition $(\mathbf{X}_{p,n}, e_p, \mathbf{X}_{p,n+1}) \in \delta_p$ is enabled, thus yielding the desired conclusion that $\alpha' = \alpha \cdot e_p \in \mathcal{L}_{\mathcal{A}(\mathcal{G})}$.

Case 2: rule *Resume* has been applied. The rule precondition implies that there exist a symbol e_p in p's buffer (formally: $e_p \in \beta_p$), where p is the process that previously issued the event symbol along the input stream $\hat{\alpha}$ at some position i. Such symbol e_p was buffered by applying rule *Delay*. Since the argumentation exposed for the previous case (rule *Pass*) does neither rely on the provenance of symbol e_p nor on the structure of the input stream $\hat{\alpha}'$, the same conclusion $\alpha' \in \mathcal{L}_{\mathcal{A}(\mathcal{G})}$ can be reached, completing the proof.

Lemma 2. *(Soundness) Let us assume any system P and any global type $\mathcal{G}$. The enforcer $E(P, \mathcal{G})$ defined as of Sec. 3.1 satisfies the following property:*

$$\mathcal{L}_{E(P,\mathcal{G})} \subseteq \mathcal{L}_{\mathcal{G}}$$

Proof. This property holds immediately since, as we clarified in this appendix, by definition $\mathcal{L}_{\mathcal{G}} = \mathcal{L}_{\mathcal{A}(\mathcal{G})}$, i.e. the language of CFSM defined as $\{\mathcal{A}(\mathcal{G}\lceil p)\}_{p \in P}$, and, by Lemma 1, $\mathcal{L}_{E(P,\mathcal{G})} = \mathcal{L}_{\mathcal{A}(\mathcal{G})}$. □

Lemma 3. *(Transparency) Let us assume any system P and any global type $\mathcal{G}$. The enforcer $E(P, \mathcal{G})$ defined as of Sec. 3.1 satisfies the following property:*

$$\forall \alpha \in \mathcal{L}_P \cap \mathcal{L}_{\mathcal{G}}.\ \alpha \in \mathcal{L}_{E(P,\mathcal{G})}$$

Proof. Since by definition $\mathcal{L}_{\mathcal{G}} = \mathcal{L}_{\mathcal{A}(\mathcal{G})}$, the premise implies $\alpha \in \mathcal{L}_{\mathcal{A}(\mathcal{G})}$. By Lemma 1, it follows that $\alpha \in \mathcal{L}_{E(P,\mathcal{G})}$. □

B Local Types for the Running Example

In Fig. 8 we report a detailed representation of local types derived by projecting global type in Fig. 4a from our running example. As already mentioned, the projection function is standard and well known from the literature. The ChoEn tool implements the description given in [16].

Let us observe that different actors have different numbers of transforms and may omit some of the state variables. This is because each interaction (i.e., message exchange) involves two participants (a sender and a receiver), while the other actors are not involved. For instance, both `Alice` and `Charlie` have the state variable `x4`: the former sends the message (cf. line 5, `x4 = Alice!cwin(int)->Charlie; x8`), while the latter receives it (cf. line 33, `x4 = Charlie?cwin(int)<-Alice; x6`). For similar reasons, both `Alice` and `Bob` have the state variable `x5`. Conversely, `Charlie` does not have the state variable `x5`, and `Bob` does not have `x4`. `Dana` has neither of these state variables, since she is not involved in either of the two interactions with `Alice`.

```
1  Alice := {
2    x0 + x18  = x1
3    x1        = x2 | x13
4    x2        = x4 (+) x5
5    x4        = Alice!cwin(int)->Charlie; x8
6    x5        = Alice!bwin(int)->Bob; x9
7    x8 + x9   = x10
8    x10       = x11 | x12
9    x11       = Alice?sig(int)<-Bob; x14
10   x12 | x13 = x15
11   x15       = Alice?msg(int)<-Charlie; x16
12   x14 | x16 = x17
13   x17       = Alice!free(int)->Dana; x18
14 } in x0
15
16 Bob := {
17   x0 + x18  = x1
18   x1        = x2 | x13
19   x2        = x6 & x5
20   x5        = Bob?bwin(int)<-Alice; x7
21   x6        = Bob?blose(int)<-Charlie; x8
22   x7        = Bob!close(int)->Charlie; x9
23   x8 + x9   = x10
24   x10       = x11 | x12
25   x11       = Bob!sig(int)->Alice; x14
26   x12 | x13 = x16
27   x14 | x16 = x18
28 } in x0
```

```
29 Charlie := {
30   x0 + x18  = x1
31   x1        = x2 | x3
32   x2        = x4 & x7
33   x4        = Charlie?cwin(int)<-Alice; x6
34   x6        = Charlie!blose(int)->Bob; x8
35   x7        = Charlie?close(int)<-Bob; x9
36   x8 + x9   = x10
37   x10       = x14 | x12
38   x12 | x13 = x15
39   x3        = Charlie!busy(int)->Dana; x13
40   x15       = Charlie!msg(int)->Alice; x16
41   x14 | x16 = x18
42 }
43
44 Dana := {
45   x0 + x18  = x1
46   x1        = x2 | x3
47   x2        = x8 & x9
48   x8 + x9   = x10
49   x10       = x14 | x12
50   x12 | x13 = x16
51   x3        = Dana?busy(int)<-Charlie; x13
52   x14 | x16 = x17
53   x17       = Dana?free(int)<-Alice; x18
54 }
```

Fig. 8: Local types for the running example

C Auxiliary Libraries

The Solidity contracts generated by the enforcer make use of some utility functions coded in Solidity.

Fig. 9 reports the code of library `Events`. In it, function `eq` tests whether the passed `Event` matches the attributes passed as parameters to the function itself, while function `contains` tests whether an array of `Event`s contains as item the `Event` passed as second argument.

Fig. 10 displays the code of the `Incomp` library, a collection of function most of which operate on the `struct IRel`. Together, the `struct` and the `function`s are used to encode the equivalence relation $\equiv_{\mathcal{T}}$ among state variables of a given local type $\mathcal{T}$.

Finally, a library named `States` implements methods for encoding a set of `STATE`s using Solidity arrays. The code is not reported for the sake of conciseness and since it is pretty straightforward.

Note that the functions of these libraries do not depend on the choreography to be enforced.

```
1  library Events {
2    function eq(Event memory ev, ACTION act, ACTOR a, ACTOR b, MSG label)
3             public pure returns (bool) {
4      return ev.action == act && ev.initiator == a && ev.target == b &&
5             ev.label == label;
6    }
7
8    function contains(Event[] memory events, Event memory ev)
9             public pure returns (bool) {
10     for (uint idx=0; idx<events.length; idx++) {
11       Event memory curr = events[idx];
12       if (curr.action == ev.action && curr.initiator == ev.initiator &&
13           curr.target == ev.target && curr.label == ev.label) {
14         return true;
15       }
16     }
17     return false;
18   }
19 }
```

Fig. 9: The library `Events`

```
library Incomp {

  function alt_pack(STATE s1, STATE s2) public pure returns (uint256) {
    if (s1 < s2) { return (uint256(s1) << 128) | uint256(s2); }
    else { return (uint256(s2) << 128) | uint256(s1); }
  }

  function add(IRel storage pr, STATE s1, STATE s2) public {
    require(s1 != s2);

    uint256 key = alt_pack(s1, s2);
    // if already exists, skip
    if (pr.pairs[key]) { return; }

    pr.pairs[key] = true;
    pr.to_prune[s1].push(s2);
    pr.to_prune[s2].push(s1);
  }

  function remove(IRel storage pr, STATE s) public {
    // remove any possible pair (s,t)
    for (uint idx=0; idx<States.COUNT; idx++) {
      STATE t = States.get(idx);
      uint256 key = alt_pack(s, t);
      delete pr.pairs[key];
    }
  }

  function update(IRel storage pr, STATE s, STATE[] memory new_alt)
    public {
    for (uint i=0; i<States.COUNT; i++) {
      STATE t = States.get(i);
      // check whether pair (s,t) or (t,s) are in the incomp IRelation
      uint key = alt_pack(s, t);
      if (pr.pairs[key]) {
        // replace (s,t) or (t,s) with (a,t) for any a in new_alt
        delete pr.pairs[key];
        for (uint j=0; j<new_alt.length; j++) {
          STATE a = new_alt[j];
          uint new_key = alt_pack(a, t);
          pr.pairs[new_key] = true;
        }
      }
    }
  }
}
```

Fig. 10: The library `Incomp` implementing operations on the incompatibility relation

D Rules Functions

Fig. 11 contains the Solidity encoding of rule *Pass*. This rule reacts to an instance of the `Event` structure representing a new request from the process asking for permission of executing an action (i.e. sending or receiving a message). This rule applies whenever the local type has rules matching one of the following forms:

```
x1 = initiator!message(sort)->target ; x2
x1 = initiator?message(sort)<-target ; x2
```

where *initiator* and *target* are instances of `ACTOR`, *message* is of type *MSG*, and *sort* are of type `SORT`.

Note that the `apply_pass` function is invoked by several contracts, each storing their own array of `STATE[]` describing the current superstate and their own instance of `IRel`, tracking the process' incompatibility relation. For this reason, rules functions have been implemented as auxiliary functions, outside the scope of any contract, and must receive as first two arguments a `STATE[]` array and an instance of `IRel`. The parameters `STATE lhs` denote instead the left-hand side state variable appearing in the rule and the attribute `STATE rhs` represents the right-hand side state variable of the rule. The `apply_pass` function is invoked multiple times inside the `check_event` function of a contract, based on the send/receive transforms appearing in the local type of the process from which the contract is generated (cf. Fig. 6, for instance). Note that the `apply_pass` function tests the preconditions of the rules, and if they are satisfied apply the side-effects of the rules on the passed instances `states` and `pr`. The way `states` is updated is mainly by replacing the state variable `lhs` in `states` at position `pos`, with state variable `rhs`. In addition, any state variable marked as *alternative* with respect to `lhs` in the incompatibility relation `pr` is collected in the auxiliary array `to_prune`, and every item in it is removed from the process' superstate `states`. This rule requires that any incompatibility $(\texttt{lhs}, x)$ for some state variable x must be replaced with incompatibility $(\texttt{rhs}, x)$. This is done invoking `Incomp.update`.

Fig. 11 also contains the Solidity encoding of rules *Fork* and *Join*. They can be considered *dual* rules, since one spawns two threads and the other reconnects them. Note that the first rule, if it finds `lhs` in the superstate `STATE[] states`, replaces it with state variables `rhs1` and `rhs2`. This also requires that any reported incompatibility $(\texttt{lhs}, x)$ for some state variable x must be replaced with incompatibility $(\texttt{rhs1}, x)$ and $(\texttt{rhs2}, x)$ in `IRel pr` (remember that the incompatibility relation is assumed to be reflexive). This is done by invoking `Incomp.update`. The second rule, instead, first look for state variables `lhs1` and `lhs2` in the passed superstate. If it finds them, it replaces them with state variable `rhs`. Next, it replaces any incompatibility $(\texttt{lhs1}, x)$ or $(\texttt{lhs2}, x)$ with $(\texttt{rhs}, x)$, invoking `Incomp.update`.

Finally, Fig. 12 contains the Solidity implementation of rules *Passthrough*, *Internal/External Choice*, *Merge-L*, and *Merge-R* (the last two in a single function).

```
function apply_pass(STATE[] storage states, IRel storage pr,
                    STATE lhs, STATE rhs) returns (bool) {

  require(lhs != rhs);
  int256 pos = States.set_find(states, lhs);
  if (pos >= 0) {
    states[uint256(pos)] = rhs;
    STATE[] memory to_prune = Incomp.alt_to(pr, lhs);
    States.set_remove(states, to_prune);
    Incomp.update(pr, lhs, States.set_of(rhs));
    return true;
  }
  return false;
}


function apply_fork(STATE[] storage states, IRel storage pr,
                    STATE lhs, STATE rhs1, STATE rhs2) returns (bool)  {
  require(lhs != rhs1); require(lhs != rhs2);
  int256 pos = States.set_find(states, lhs);

  if (pos >= 0) {
    states[uint256(pos)] = rhs1;
    states.push(rhs2);
    Incomp.update(pr, lhs, States.set_of(rhs1, rhs2));
    return true;
  }
  return false;
}

function apply_join(STATE[] storage states, IRel storage pr,
                    STATE lhs1, STATE lhs2, STATE rhs) returns (bool)  {
  require(lhs1 != rhs); require(lhs2 != rhs);
  int256 pos1 = States.set_find(states, lhs1);

  if (pos1 >= 0) {
    int256 pos2 = States.set_find(states, lhs2);
    if (pos2 >= 0) {
      states[uint256(pos1)] = rhs;
      states[uint256(pos2)] = states[states.length - 1];
      states.pop();
      Incomp.update(pr, lhs1, States.set_of(rhs));
      Incomp.update(pr, lhs2, States.set_of(rhs));
      return true;
    }
  }
  return false;
}
```

Fig. 11

```
function apply_passthrough(STATE[] storage states, IRel storage pr,
                           STATE lhs, STATE rhs)  returns (bool) {
  require(lhs != rhs);
  int256 pos = States.set_find(states, lhs);
  if (pos >= 0) {
    states[uint256(pos)] = rhs;
    Incomp.update(pr, lhs, States.set_of(rhs));
    return true;
  }
  return false;
}

function apply_choice(STATE[] storage states, IRel storage pr,
                      STATE lhs, STATE rhs1, STATE rhs2) returns (bool) {
  require(lhs != rhs1);
  require(lhs != rhs2);
  int256 pos = States.set_find(states, lhs);

  if (pos >= 0) {
    states[uint256(pos)] = rhs1;
    states.push(rhs2);

    Incomp.add(pr, rhs1, rhs2);
    Incomp.update(pr, lhs, States.set_of(rhs1, rhs2));
    return true;
  }
  return false;
}

function apply_merge(STATE[] storage states, IRel storage pr,
                     STATE lhs1, STATE lhs2, STATE rhs) returns (bool) {
  require(lhs1 != rhs);
  require(lhs2 != rhs);

  if (apply_passthrough(states, pr, lhs1, rhs)) {
    return true;
  } else if (apply_passthrough(states, pr, lhs2, rhs)) {
    return true;
  }
  return false;
}
```

Fig. 12